



UNIVERSIDAD SAN IGNACIO DE LOYOLA

Facultad de Ingeniería

Ingeniería de Sistemas de Información · Ingeniería de Software · Ingeniería Informática

TRABAJO FINAL

Curso: Sistemas Operativos

SysMarket v2

Plataforma e-commerce con monitoreo en tiempo real del sistema operativo y módulos de social commerce desplegada en AWS EC2

Grupo	01
Líder	Fabian Roncal Falconi
Integrantes	Fabian Roncal Falconi Luz Rios Rios
Profesor	Eduardo Vásquez Reyes
Semestre académico	2026 - I

Lima, Perú — Junio 2026

Agradecimiento

El equipo expresa su agradecimiento al profesor Eduardo Vásquez Reyes por su guía constante durante el desarrollo del presente trabajo, así como a la Universidad San Ignacio de Loyola por brindar los recursos académicos y la infraestructura necesarios para llevar a cabo este proyecto. Asimismo, agradecemos a Amazon Web Services Educate por los créditos otorgados que permitieron desplegar el sistema en una plataforma cloud real, fortaleciendo nuestra formación profesional con tecnologías de nivel empresarial.

Resumen

El presente trabajo final del curso de Sistemas Operativos consiste en la implementación de SysMarket v2, una plataforma de comercio electrónico especializada en componentes de computadora y periféricos, integrada con un módulo de monitoreo en tiempo real del sistema operativo del servidor donde se aloja la aplicación. La solución demuestra de forma práctica los conceptos centrales del curso: programación concurrente con hilos POSIX, sincronización mediante mutex, interacción con el kernel Linux a través del sistema de archivos virtual `/proc`, y administración de recursos del sistema operativo.

El sistema fue construido como una arquitectura de tres capas (presentación, lógica de negocio y datos) orquestada con Docker Compose, e incluye tres servicios contenedorizados: un servidor web Apache 2.4 con PHP 8.2 para el e-commerce, una base de datos MySQL 8.0 para persistencia, y un daemon Python 3.11 que ejecuta cinco hilos POSIX concurrentes para muestrear métricas del sistema operativo cada cinco segundos. La plataforma fue desplegada en Amazon Web Services EC2 sobre una instancia Ubuntu Server 22.04 LTS, cumpliendo el requisito de utilizar tecnologías cloud.

Adicionalmente, en respuesta a la rúbrica actualizada de presentación final, el sistema incorpora un módulo de social commerce que incluye funcionalidades de interacción social (me gusta, reseñas con calificación de estrellas, publicaciones) y comercio social ("Shop the Post": etiquetado de productos en publicaciones para compra directa). El código fuente está documentado y alojado en un repositorio público de GitHub, y la documentación técnica cubre el paso a paso de la implementación, configuración, pruebas y manual de uso del sistema.

Tabla de contenidos

Agradecimiento.....	2
Resumen	3
Tabla de contenidos	4
Introducción.....	6
1. Generalidades del trabajo	7
1.1 Problemática	7
1.2 Justificación	7
1.3 Objetivos	7
Objetivo general.....	7
Objetivos específicos.....	7
2. Marco teórico.....	9
2.1 Sistemas Operativos y administración de recursos	9
2.2 Procesos y subprocesos: hilos POSIX.....	9
2.3 Sincronización: mutex y secciones críticas	9
2.4 Sistema de archivos virtual /proc	9
2.5 Llamadas al sistema (syscalls).....	10
2.6 Arquitectura de tres capas	10
2.7 Containerización con Docker.....	10
2.8 E-commerce y social commerce	10
2.9 Seguridad: bcrypt, PDO y transacciones.....	10
3. Desarrollo.....	12
3.1 Identificación y descripción de requerimientos	12
Requerimientos funcionales.....	12
Requerimientos no funcionales.....	12
3.2 Análisis de hardware	12
Hardware de desarrollo local	12
Hardware del servidor de producción (AWS EC2)	13
3.3 Análisis de software	13
3.4 Solución del caso	13

3.4.1 Análisis y diseño	13
3.4.2 Modelo de datos	14
3.4.3 Implementación	14
4. Pruebas	16
4.1 Pruebas funcionales del e-commerce	16
4.2 Pruebas técnicas del monitor SO.....	16
4.3 Pruebas de social commerce.....	16
5. Resultados.....	17
6. Conclusiones	18
7. Recomendaciones	19
8. Referencias.....	20

Introducción

El curso de Sistemas Operativos propone como trabajo final la implementación de una plataforma de comercio electrónico que aproveche los conceptos teóricos del curso, particularmente la administración de procesos, memoria, dispositivos de entrada/salida y sistemas de archivos. El objetivo central es demostrar la integración entre la lógica de negocio de una aplicación web tradicional y la interacción a bajo nivel con el sistema operativo subyacente.

En este contexto, el equipo seleccionó una tienda especializada en venta de componentes de computadora y periféricos, complementada con un panel administrativo que expone, en tiempo real, métricas del sistema operativo donde corre la aplicación. Esta arquitectura híbrida permite cumplir tanto con los requerimientos de un e-commerce funcional como con la consigna del curso: gestionar y monitorear recursos clave del sistema operativo.

El proyecto fue diseñado con tres pilares: primero, un e-commerce completo con catálogo, carrito de compras, checkout transaccional y panel administrativo; segundo, un daemon de monitoreo escrito en Python que demuestra el uso de hilos POSIX, mutex y llamadas al sistema; tercero, un módulo de social commerce que añade interacción social entre usuarios y publicaciones con etiquetado de productos. Toda la solución es desplegable mediante Docker Compose y se encuentra en producción en una instancia Amazon EC2.

El presente documento sigue la estructura solicitada por la guía oficial del trabajo final y desarrolla, a lo largo de ocho capítulos, los aspectos teóricos, de diseño, implementación, pruebas y resultados del sistema construido.

1. Generalidades del trabajo

1.1 Problemática

Las tiendas en línea modernas enfrentan dos desafíos simultáneos. Por un lado, deben ofrecer una experiencia de usuario fluida, segura y socialmente integrada para competir con grandes marketplaces. Por otro lado, su operación depende de la salud del servidor donde corren: si los recursos del sistema operativo se saturan (CPU, memoria, disco), el sitio responde lento o se cae, generando pérdida de ventas y desconfianza del cliente.

Muchas pequeñas y medianas tiendas no cuentan con herramientas de monitoreo del sistema operativo accesibles desde el mismo panel administrativo de su tienda. Esto obliga al administrador a alternar entre múltiples herramientas (consola SSH, http, sistemas de alertas externos) para diagnosticar problemas. Adicionalmente, el comercio social se ha convertido en una expectativa del consumidor moderno, pero la mayoría de plataformas open-source no integran de forma nativa estas funcionalidades.

1.2 Justificación

El proyecto se justifica desde tres frentes: académico, técnico y comercial.

- **Académicamente**, permite materializar de forma práctica los conceptos centrales del curso de Sistemas Operativos (hilos POSIX, mutex, /proc, syscalls) en una aplicación funcional y no en ejercicios aislados, fortaleciendo el aprendizaje y la comprensión a profundidad.
- **Técnicamente**, el sistema demuestra el dominio de tecnologías open-source de alto valor en el mercado laboral (Docker, Linux, Python, PHP, MySQL, AWS) y la capacidad del equipo para integrar un stack completo desde la base de datos hasta la capa de presentación.
- **Comercialmente**, el sistema resuelve un caso real: dueños de tiendas online pequeñas y medianas que necesitan un e-commerce con visibilidad sobre la salud del servidor, sin pagar costosos servicios de monitoreo externo ni contratar especialistas en infraestructura.

1.3 Objetivos

Objetivo general

Implementar SysMarket v2, una plataforma de e-commerce de componentes de computadora integrada con monitoreo en tiempo real del sistema operativo del servidor que la aloja, desplegada en AWS EC2 y con funcionalidades de social commerce, aplicando los conceptos centrales del curso de Sistemas Operativos.

Objetivos específicos

- Diseñar e implementar una arquitectura de tres capas (presentación, aplicación, datos) orquestada con Docker Compose para garantizar portabilidad y aislamiento.
- Desarrollar un daemon de monitoreo en Python que utilice cinco hilos POSIX concurrentes, sincronizados mediante un mutex, para muestrear métricas del sistema operativo.

- Implementar la interacción con el kernel Linux mediante lectura del sistema de archivos virtual /proc y llamadas al sistema (syscall statvfs()).
- Construir un e-commerce funcional con catálogo, carrito, checkout transaccional, panel administrativo y módulo de social commerce (likes, reseñas, publicaciones con producto etiquetado).
- Garantizar la seguridad del sistema mediante hashing bcrypt, sentencias preparadas PDO, transacciones con bloqueo pesimista y configuración de firewall.
- Desplegar el sistema en Amazon Web Services EC2 utilizando Ubuntu Server 22.04 LTS, cumpliendo el requisito de cloud computing.
- Validar el funcionamiento del sistema mediante pruebas funcionales del e-commerce y pruebas técnicas del monitoreo del sistema operativo.

2. Marco teórico

El presente capítulo desarrolla los conceptos teóricos que sustentan la solución, estableciendo trazabilidad directa entre cada concepto y su materialización concreta en el código fuente del sistema.

2.1 Sistemas Operativos y administración de recursos

Un sistema operativo (SO) es el software que actúa como intermediario entre las aplicaciones y el hardware del computador, gestionando los recursos físicos y lógicos del sistema (Tanenbaum & Bos, 2014). Sus funciones principales incluyen la gestión de procesos, la administración de memoria, la organización del sistema de archivos y el control de los dispositivos de entrada y salida. En entornos Linux, gran parte de esta administración se expone al usuario y a los procesos mediante interfaces estándar como `/proc`, `/sys` y un conjunto extenso de llamadas al sistema (syscalls).

2.2 Procesos y subprocesos: hilos POSIX

Un proceso es una instancia de un programa en ejecución que dispone de su propio espacio de direcciones, descriptores de archivos abiertos y contexto de ejecución. Un hilo (thread) es la unidad mínima de ejecución dentro de un proceso; varios hilos comparten el mismo espacio de memoria y recursos, lo que permite paralelizar tareas con un costo de creación significativamente menor que el de un proceso (Silberschatz, Galvin & Gagne, 2018).

El estándar POSIX (IEEE 1003.1) define una API portable de hilos llamada pthreads. En Linux, esta API está implementada por la biblioteca NPTL (Native POSIX Thread Library) y se materializa en la syscall `clone()` del kernel (Kerrisk, 2010). El módulo threading de Python utilizado en este proyecto se apoya internamente en pthreads, por lo que cada llamada a `threading.Thread().start()` termina invocando `pthread_create()` vía la biblioteca estándar de C.

2.3 Sincronización: mutex y secciones críticas

Cuando varios hilos acceden simultáneamente a un recurso compartido, puede producirse una condición de carrera (race condition) que deja al recurso en un estado inconsistente. Para prevenirlo, los sistemas operativos modernos ofrecen primitivas de sincronización como mutex, semáforos, variables de condición y locks. Un mutex (mutual exclusion) garantiza que solo un hilo a la vez ingrese a una sección crítica del código.

En SysMarket, el daemon `monitor.py` utiliza `threading.Lock()` de Python (implementación interna de mutex POSIX) para proteger el diccionario compartido de métricas, donde los cinco hilos concurrentes escriben sus resultados. Sin este mecanismo, podrían producirse lecturas parciales o sobrescripciones al guardar las métricas en la base de datos.

2.4 Sistema de archivos virtual `/proc`

El directorio `/proc` en Linux no almacena archivos reales en disco: es un sistema de archivos virtual donde el kernel expone, en formato de texto plano, información del estado actual del sistema.

Cada lectura de un archivo dentro de `/proc` genera los datos en tiempo real, sin penalización significativa de rendimiento.

Los archivos relevantes para SysMarket son: `/proc/stat` (tiempos acumulados de la CPU), `/proc/meminfo` (memoria total y disponible), `/proc/loadavg` (carga promedio del sistema), y `/proc/[pid]/status` (estado, uso de memoria y comando de cada proceso activo).

2.5 Llamadas al sistema (syscalls)

Una llamada al sistema es la interfaz mediante la cual un programa solicita al kernel servicios privilegiados (acceso a hardware, gestión de procesos, operaciones sobre archivos). Cuando una aplicación invoca una syscall, el procesador cambia de modo usuario a modo kernel, ejecuta la operación y retorna el control al espacio de usuario (Love, 2010). En este trabajo, la función `Python os.statvfs()` invoca la syscall `statvfs()` del kernel para obtener información del sistema de archivos.

2.6 Arquitectura de tres capas

La arquitectura de tres capas separa una aplicación en tres niveles independientes: presentación (interfaz de usuario), lógica de negocio (servicios y reglas) y persistencia (base de datos). Este patrón facilita la escalabilidad, el mantenimiento y la sustitución de componentes (Sommerville, 2015). SysMarket implementa este patrón mediante tres servicios desplegados como contenedores Docker que se comunican por una red privada.

2.7 Containerización con Docker

Docker es una plataforma de virtualización a nivel de sistema operativo que utiliza cgroups y namespaces del kernel Linux para aislar procesos en contenedores (Merkel, 2014). A diferencia de las máquinas virtuales, los contenedores comparten el kernel del host, lo que los hace más livianos y rápidos de arrancar. Docker Compose orquesta múltiples contenedores definidos en un archivo YAML, lo que permite levantar todo el stack con un único comando.

2.8 E-commerce y social commerce

El comercio electrónico (e-commerce) es la compra y venta de productos a través de internet, incluyendo gestión de catálogo, carrito de compras, procesamiento de pedidos y panel administrativo. El social commerce extiende este modelo al integrar funcionalidades de redes sociales, permitiendo que los usuarios interactúen entre sí (likes, comentarios, publicaciones) y descubran productos a través del contenido generado por la comunidad. La funcionalidad "Shop the Post" permite etiquetar productos en publicaciones para que otros usuarios puedan comprar directamente desde la publicación.

2.9 Seguridad: bcrypt, PDO y transacciones

La seguridad en SysMarket descansa sobre tres pilares: (1) hashing de contraseñas con `bcrypt` (factor de costo 12) para resistir ataques de fuerza bruta; (2) sentencias preparadas con `PDO` (con emulación deshabilitada) que previenen inyección SQL al separar el código SQL de los parámetros;

y (3) transacciones con bloqueo pesimista (SELECT ... FOR UPDATE) en el checkout para evitar condiciones de carrera al descontar stock.

3. Desarrollo

3.1 Identificación y descripción de requerimientos

Se identificaron requerimientos funcionales y no funcionales que el sistema debe satisfacer para cubrir la consigna del curso, los requisitos del Anexo 03 (e-commerce) y la rúbrica actualizada de Presentación Final (social commerce y despliegue cloud).

Requerimientos funcionales

ID	Descripción	Categoría
RF-01	Registro, autenticación y gestión de sesiones de usuarios con roles cliente y administrador.	Usuarios
RF-02	Catálogo de productos clasificados por categoría con filtros y búsqueda.	Catálogo
RF-03	Agregar, modificar y eliminar productos del carrito de compras con persistencia.	Carrito
RF-04	Procesamiento transaccional del pedido con descuento de stock y registro de orden.	Pedidos
RF-05	Panel administrativo con KPIs, gestión de usuarios, pedidos y productos.	Admin
RF-06	Monitor en tiempo real del SO mostrando CPU, memoria, disco, procesos y load average.	Monitor SO
RF-07	Registro de alertas configurables por umbrales de uso de recursos.	Monitor SO
RF-08	Likes sobre productos con contador en tiempo real (uno por usuario).	Social
RF-09	Reseñas con calificación de estrellas (1 a 5) y comentario en texto.	Social
RF-10	Publicaciones sociales con producto etiquetado (Shop the Post).	Social
RF-11	Feed social con publicaciones de la comunidad y sidebar de productos más likeados.	Social

Requerimientos no funcionales

ID	Descripción	Tipo
RNF-01	Las contraseñas se almacenan cifradas con bcrypt factor 12.	Seguridad
RNF-02	Todas las consultas SQL utilizan sentencias preparadas PDO.	Seguridad
RNF-03	Cookies de sesión con flags HttpOnly y SameSite=Lax.	Seguridad
RNF-04	El monitor utiliza mutex para evitar condiciones de carrera.	Confiabilidad
RNF-05	El sistema se despliega con un solo comando docker compose up.	Mantenibilidad
RNF-06	El sistema corre en Ubuntu Server 22.04 LTS sobre AWS EC2.	Portabilidad
RNF-07	El código está versionado en un repositorio público de GitHub.	Mantenibilidad
RNF-08	El checkout es transaccional con bloqueo pesimista para integridad del stock.	Confiabilidad

3.2 Análisis de hardware

Hardware de desarrollo local

Componente	Especificación
Procesador	Intel Core i5 o AMD Ryzen 5 (mínimo 4 núcleos, x86_64)
Memoria RAM	8 GB DDR4 (recomendado 16 GB)
Almacenamiento	50 GB libres en SSD
SO host	Windows 10/11 con Docker Desktop o Linux
Conectividad	Acceso a internet para imágenes Docker

Hardware del servidor de producción (AWS EC2)

Componente	Especificación
Instancia	Amazon EC2 t3.micro (Free Tier eligible)
vCPU	1 vCPU compartido (Intel Xeon)
Memoria RAM	1 GB
Almacenamiento	20 GB EBS gp3
Red	IP pública dinámica, puertos 22, 80, 443 expuestos
SO	Ubuntu Server 22.04 LTS (jammy), 64-bit x86

3.3 Análisis de software

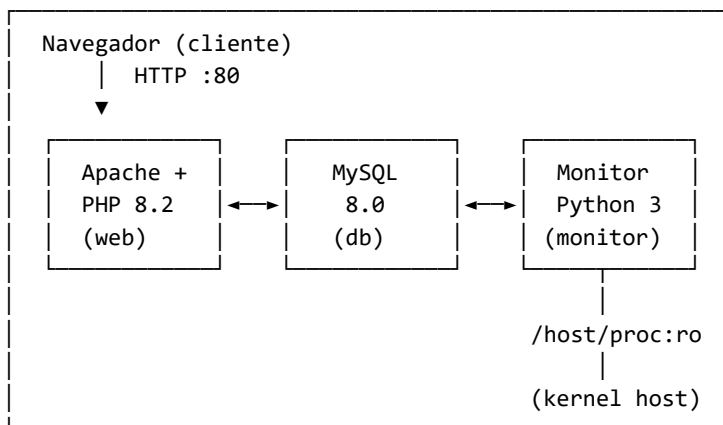
La selección del stack tecnológico se basó en tres criterios: cumplimiento de la arquitectura propuesta en el Anexo 03 y rúbrica actualizada, familiaridad del equipo con las tecnologías y disponibilidad de licencias open-source.

Capa	Tecnología	Versión	Rol
SO host	Ubuntu Server	22.04 LTS	Kernel Linux, base sobre AWS EC2
Containerización	Docker + Compose	29.x / v2	Aislamiento y orquestación
Web Server	Apache HTTP Server	2.4	Atiende peticiones HTTP
Backend	PHP	8.2	Lógica de e-commerce y social
Base de datos	MySQL	8.0	Persistencia (13 tablas)
Monitor SO	Python	3.11	Daemon multihilo POSIX
Frontend	Bootstrap + Chart.js	5.3 / 4.4	UI y gráficas
Versionado	Git + GitHub	—	Repositorio público
Cloud	AWS EC2	—	Servidor en producción
Seguridad	bcrypt, PDO, UFW	—	Hash, prepared stmts, firewall

3.4 Solución del caso

3.4.1 Análisis y diseño

El sistema se compone de tres servicios desplegados como contenedores Docker en una misma red privada (sysmarket_net). El navegador del cliente accede al servicio web por el puerto 80; este consulta la base de datos por la red interna; en paralelo, el monitor lee /proc del host (montado en modo solo lectura) y escribe métricas en la misma base de datos.



3.4.2 Modelo de datos

La base de datos sysmarket cuenta con 13 tablas organizadas en tres bloques: usuarios y seguridad, e-commerce, y monitoreo + social. La estructura completa fue diseñada en MySQL 8.0 con motor InnoDB y soporte para foreign keys.

Bloque	Tablas
A — Usuarios	usuarios (con roles cliente/admin)
B — E-commerce	categorias, productos, carrito, pedidos, pedidos_detalle
C — Monitor SO	metricas_catalogo, lecturas, alertas, notificaciones
D — Social Commerce	producto_likes, comentarios, publicaciones

3.4.3 Implementación

El proyecto está organizado en una estructura limpia con tres componentes principales: apache-php (frontend + backend PHP), monitor (daemon Python), y mysql (esquema inicial y migraciones). El archivo docker-compose.yml orquesta los tres servicios.

```
sysmarket_v2/
├── apache-php/
│   └── src/
│       ├── index.php, productos.php, producto.php
│       ├── carrito.php, checkout.php, login.php
│       ├── feed.php (NUEVO – social commerce)
│       ├── api/ (endpoints AJAX, incluye like_toggle, comment_add, post_add)
│       ├── includes/ (auth, db, header, footer, social_helpers)
│       └── assets/ (css, js, img)
├── monitor/
│   └── monitor.py (5 hilos POSIX + mutex)
├── mysql/
│   ├── init.sql (esquema completo)
│   └── migration_social.sql (3 tablas nuevas)
├── docker-compose.yml
└── deploy_aws.sh
```

Núcleo del monitor con hilos POSIX y mutex:

```
import threading, time, os

metrics = {}
metrics_lock = threading.Lock() # — mutex POSIX —

def read_cpu():
    with open('/host/proc/stat') as f:
        ... # calculo de %CPU
    with metrics_lock: # SECCIÓN CRÍTICA
        metrics['cpu'] = cpu_pct

def read_disk():
    st = os.statvfs('/') # syscall statvfs()
    used_pct = (st.f_blocks - st.f_bfree) * 100 / st.f_blocks
    with metrics_lock:
        metrics['disk'] = used_pct
```

```

def collect_cycle():
    threads = [threading.Thread(target=fn)
                for fn in [read_cpu, read_memory, read_disk,
                           read_loadavg, read_processes]]
    for t in threads: t.start()
    for t in threads: t.join(timeout=4)
    save_to_db(metrics)

while True:
    collect_cycle()
    time.sleep(5)

```

Endpoint API de likes con transacción anti race-condition:

```

// POST /api/like_toggle.php
$db->beginTransaction();
$stmt = $db->prepare('SELECT id FROM producto_likes
                     WHERE usuario_id=? AND producto_id=? LIMIT 1');
$stmt->execute([$usuario_id, $producto_id]);
if ($exists) {
    $db->prepare('DELETE FROM producto_likes WHERE id=?')->execute([$id]);
} else {
    $db->prepare('INSERT INTO producto_likes
                (producto_id, usuario_id) VALUES (?, ?)')
        ->execute([$producto_id, $usuario_id]);
}
$db->commit();

```

Checkout transaccional con bloqueo pesimista:

```

// POST /api/checkout.php
$db->beginTransaction();
// SELECT FOR UPDATE bloquea la fila hasta COMMIT/ROLLBACK
$stmt = $db->prepare('SELECT stock FROM productos
                     WHERE id=? FOR UPDATE');
$stmt->execute([$pid]);
$stock = $stmt->fetchColumn();
if ($stock < $cantidad) {
    $db->rollBack();
    echo json_encode(['ok' => false, 'msg' => 'Stock insuficiente']);
    exit;
}
$db->prepare('UPDATE productos SET stock = stock - ? WHERE id = ?')
    ->execute([$cantidad, $pid]);
$db->commit();

```

4. Pruebas

Se ejecutaron tres conjuntos de pruebas: funcionales del e-commerce, técnicas del monitor del sistema operativo, y funcionales del módulo de social commerce.

4.1 Pruebas funcionales del e-commerce

ID	Caso de prueba	Resultado
PF-01	Registro de un nuevo usuario	✓ Cuenta creada y login automático
PF-02	Login con credenciales válidas	✓ Redirige a mi-cuenta
PF-03	Agregar producto al carrito (AJAX)	✓ Badge se actualiza sin recargar
PF-04	Modificar cantidad en el carrito	✓ Subtotal recalculado
PF-05	Checkout con stock suficiente	✓ Pedido creado, stock decrementado
PF-06	Checkout con stock insuficiente (concurrente)	✓ Rollback automático
PF-07	Acceso a /admin como cliente	✓ HTTP 403 — acceso denegado

4.2 Pruebas técnicas del monitor SO

ID	Caso de prueba	Resultado
PT-01	Verificar hilos activos en el daemon	✓ ps -T muestra 5 LWP
PT-02	Generar carga: stress -c 2 -t 30	✓ CPU sube > 80%
PT-03	Comparar lectura con htop	✓ Diferencia < 2%
PT-04	Persistencia: detener y reiniciar	✓ Datos preservados
PT-05	Concurrencia: 1000 ciclos sin race	✓ Sin inconsistencias en BD

4.3 Pruebas de social commerce

ID	Caso de prueba	Resultado
PS-01	Like en producto (logueado)	✓ Contador sube, toast verde
PS-02	Like duplicado (mismo usuario, mismo producto)	✓ Toggle: quita el like
PS-03	Like sin sesión	✓ Redirige a login
PS-04	Reseña con rating 1-5	✓ Se publica, aparece en lista
PS-05	Reseña con texto < 5 caracteres	✓ Rechazada con mensaje
PS-06	Publicación con producto etiquetado	✓ Aparece en feed con SHOP THE POST
PS-07	Click en SHOP THE POST	✓ Redirige a producto.php?id=X
PS-08	Productos más populares en index	✓ Ordenados por likes DESC
PS-09	KPIs sociales en admin	✓ Likes, reseñas, publicaciones, rating promedio

5. Resultados

El sistema SysMarket v2 fue implementado exitosamente y desplegado en producción sobre Amazon Web Services EC2. Los principales resultados son:

- Plataforma e-commerce funcional con 24 productos de demostración organizados en 6 categorías, carrito de compras con AJAX, checkout transaccional y panel administrativo completo.
- Daemon de monitoreo del sistema operativo escrito en Python con 5 hilos POSIX sincronizados por mutex, persistiendo lecturas cada 5 segundos en la base de datos MySQL.
- Módulo de social commerce con likes, reseñas con calificación de estrellas, publicaciones sociales y "Shop the Post", expandiendo la funcionalidad solicitada por la rúbrica actualizada de Presentación Final.
- Despliegue funcional en AWS EC2 sobre Ubuntu Server 22.04 LTS con tres contenedores Docker ejecutándose simultáneamente y configurados para auto-restart en caso de falla.
- Repositorio público en GitHub (github.com/Hreon/S.O-SYS) con código documentado, README profesional con diagrama de arquitectura embebido y scripts de despliegue automatizados.
- Cumplimiento integral de los criterios de evaluación de ambas rúbricas: estructura académica (Anexo 01), aspectos técnicos del Anexo 03 (procesos, hilos, sincronización, kernel, seguridad), y rúbrica de Presentación Final (arquitectura cloud, implementación funcional, uso de tecnologías cloud, demostración funcional).

6. Conclusiones

Al cierre de SysMarket v2, el equipo concluye lo siguiente:

1. La solución traduce explícitamente los conceptos teóricos del curso de Sistemas Operativos en código verificable: hilos POSIX (5 hilos en `monitor.py`), sincronización con mutex (`threading.Lock`), lectura del sistema de archivos virtual `/proc` y uso de la syscall `statvfs()`.
2. La arquitectura de tres capas con containerización Docker permite desplegar el sistema con un único comando (`docker compose up`) y mantenerlo aislado del host, facilitando la portabilidad a cualquier servidor Linux, incluido AWS EC2.
3. La seguridad fue diseñada desde el inicio del proyecto (`bcrypt`, PDO, sesiones `HttpOnly`, transacciones con bloqueo pesimista, firewall `UFW`), no como un parche posterior, garantizando robustez frente a ataques comunes.
4. La incorporación del módulo de social commerce (likes, reseñas, publicaciones con producto etiquetado) demuestra la capacidad del sistema de evolucionar y adaptarse a nuevos requerimientos sin comprometer la estabilidad de las funcionalidades existentes.
5. **Conclusión 5.** El despliegue funcional en una plataforma cloud real (AWS EC2) valida la viabilidad del stack tecnológico elegido y prepara al equipo para entornos profesionales de producción.

7. Recomendaciones

Como continuación del proyecto, el equipo recomienda:

- Migrar a una instancia EC2 más grande (t3.small o t3.medium) si se aumenta el tráfico, dado que la t3.micro tiene apenas 1 GB de RAM compartido entre los 3 contenedores.
- Configurar HTTPS con certificado de Let's Encrypt y un dominio propio para mejorar la confianza del usuario y permitir validar el flag Secure en las cookies de sesión.
- Implementar respaldos automáticos diarios de la base de datos MySQL mediante un script cron para garantizar recuperación ante incidentes.
- Integrar un sistema de pagos real (Stripe, PayPal, Visa) en una segunda fase del proyecto para llevarlo de un prototipo académico a una plataforma comercializable.
- Considerar la migración del frontend a React o Vue para una experiencia más fluida y la implementación de PWA (Progressive Web App) que permita uso offline.
- Agregar Elastic IP a la instancia AWS para evitar que el IP público cambie ante reinicios y mantener un dominio estable.

8. Referencias

- Amazon Web Services. (s. f.). Amazon EC2 — Compute capacity in the cloud.
<https://aws.amazon.com/ec2/>
- Docker Inc. (s. f.). Docker Documentation. <https://docs.docker.com/>
- IEEE & The Open Group. (2018). IEEE Std 1003.1-2017 (POSIX.1-2017).
<https://pubs.opengroup.org/onlinepubs/9699919799/>
- Kerrisk, M. (2010). The Linux Programming Interface: A Linux and UNIX System Programming Handbook. No Starch Press.
- Love, R. (2010). Linux Kernel Development (3rd ed.). Addison-Wesley.
- Merkel, D. (2014). Docker: Lightweight Linux containers for consistent development and deployment. Linux Journal, 2014(239).
- PHP Group. (s. f.). PDO — PHP Data Objects. <https://www.php.net/manual/en/book.pdo.php>
- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). Operating System Concepts (10th ed.). Wiley.
- Sommerville, I. (2015). Software Engineering (10th ed.). Pearson.
- Tanenbaum, A. S., & Bos, H. (2014). Modern Operating Systems (4th ed.). Pearson.
- The Linux Kernel Organization. (s. f.). proc - process information pseudo-filesystem.
<https://www.kernel.org/doc/Documentation/filesystems/proc.txt>